Campus Event Management Platform

A Flask and SQLite web application
with profiling-driven optimisation

Prajyot

MC25B012

Course: CS 202C

Indian Institute of Technology Dharwad

November 2026

Abstract

This report describes the Campus Event Management Platform, a web application in which students browse, search and register for campus events and administrators manage events, venues, participant lists and reports. It is built with Python and Flask, uses SQLite through raw SQL, is styled with hand-written HTML and CSS, is automated with Bash scripts and versioned with Git. The application is covered by 45 automated pytest tests. A deliberately naive first version of the reports page was profiled with `cProfile` on 2000 events and 67,423 registrations: 99% of the time was spent in 6000 small SQL queries (the *N+1 query problem*). Rewriting each report as one `JOIN` + `GROUP BY` query and adding three indexes reduced the total time from 10.9 s to 32.6 ms (about 335× faster) and the number of SQL statements from 6016 to 6, while the tests confirmed that every report number stayed the same.

Contents

1 Introduction and objectives **3**

2 Course modules used **3**

3 System architecture **3**

4 Database design **5**

5 Features and implementation **6**
5.1 Authentication and roles 6
5.2 Browsing and searching events 6
5.3 Registering and cancelling 7
5.4 Admin: venues, events and the clash check 8
5.5 Participants and CSV export 8
5.6 Reports 8

6 Bash automation **9**

7 Testing **10**

8 Profiling and optimisation **12**
8.1 Method 12
8.2 The bottleneck: the N+1 query problem 12
8.3 The fixes 14
8.4 Results 15
8.5 Query plans 16
8.6 Why it got faster, and what did not change 17

9 Git workflow **17**

10 AI usage declaration **18**

11 Challenges **19**

12 Conclusion and future work **19**

1 Introduction and objectives

Colleges run many events: technical talks, cultural nights, sports matches, workshops and seminars. Information about them is often scattered across posters and chat groups, sign-ups are collected in spreadsheets, and nobody knows how full an event is or whether two events have been booked into the same hall at the same time.

The goal of this project is a small but complete web application that solves these problems, built so that every line can be explained. The objectives were:

1. Let **students** create an account, browse upcoming events, search and filter them, and register or cancel, with the app enforcing capacity limits.
2. Let **admins** create, edit and delete events and venues, with the app rejecting **venue time clashes**, and view and export participant lists.
3. Give admins a **reports** page: registrations and fill rate per event, popular categories and venue utilisation.
4. Use at least five course modules (HTML/CSS, Python, SQL, Git, Bash), plus LaTeX for this report and the slides.
5. Back the code with **automated tests** and use a **profiler** to find and fix a real performance bottleneck, with measured before/after results.
6. Keep the code simple and well commented, with no ORM and no CSS framework, so that the SQL and CSS are written and understood by hand.

2 Course modules used

Module	Where and how it is used
HTML/CSS	15 Jinja2 templates in `app/templates/`: one `base.html` layout and 14 pages that extend it layout; one hand-written stylesheet `app/static/style.css` using CSS custom properties (variables), flexbox, CSS grid and horizontally scrolling tables for phones. The fill-rate bars on the reports page are pure CSS.
Python	Flask application factory, three Blueprints (`auth`, `events`, `admin`), Flask-Login sessions, Werkzeug password hashing, the `csv` module for export, a Faker-based seed command, pytest tests, and the `cProfile`/`pstats` profiling scripts.
SQL	Raw SQL through Python's built-in `sqlite3` module: table definitions with `PRIMARY KEY`, `UNIQUE`, `CHECK`, `FOREIGN KEY` and `ON DELETE CASCADE`; parameterised queries with `?`; `LEFT JOIN`, `GROUP BY`, `COUNT(DISTINCT)`, sub-queries and derived tables; indexes; `EXPLAIN QUERY PLAN`.
Git	23 small commits (before this documentation phase) following a `type: message` convention; `.gitignore` and `.gitattributes`.
Bash	Seven scripts in `scripts/` for setup, running, seeding, testing, profiling and building the documents, all with `set -euo pipefail` and working in Git Bash on Windows as well as on Linux/macOS.
LaTeX	This report (TikZ diagrams [14], `listings` code, a `pgfplots` chart) and the Beamer slides.

Table 1: Course modules and where they appear in the project.

3 System architecture

The application follows the classic three-tier pattern: the browser sends HTTP requests, Flask [1] decides which Python function handles each URL, the function reads or writes the SQLite database with SQL [6], and the result is rendered into HTML by a Jinja2 template. Figure 1 shows the parts.

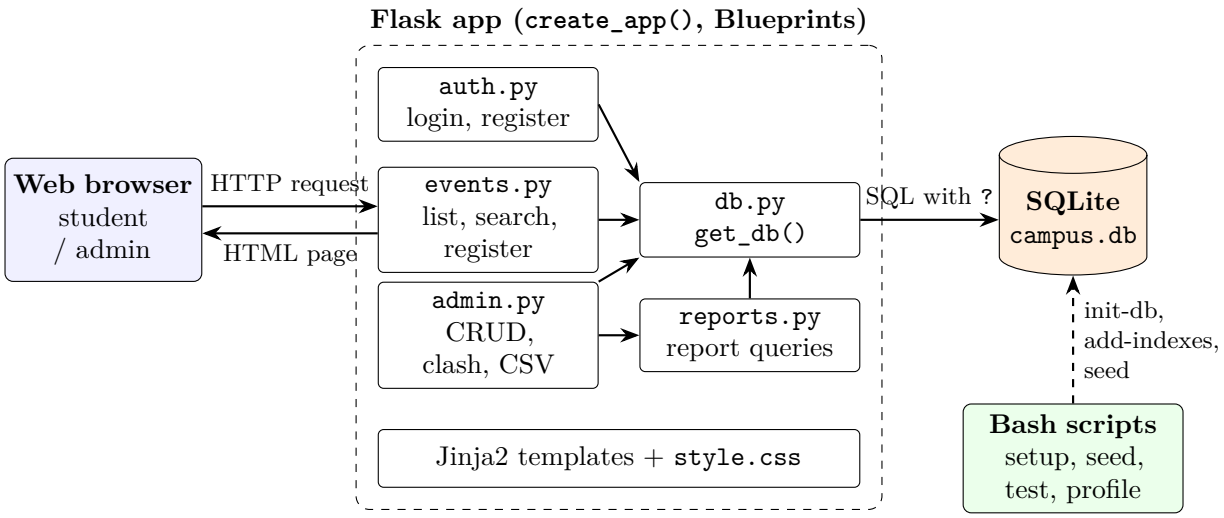


Figure 1: Architecture: browser → Flask Blueprints → SQLite.

Application factory. `create_app()` in `app/__init__.py` builds the Flask app, sets the secret key and database path, registers the Blueprints, sets up Flask-Login [2] and adds friendly 403/404 error pages. Building the app inside a function lets the tests create a fresh app that points at a temporary database.

Blueprints. Each Blueprint groups related routes in its own file: `auth` (`/auth/...`), `events` (`/events/...` and `/my-registrations`) and `admin` (`/admin/...`). Calculations that need to be tested or profiled on their own, such as `search_events()`, `find_clash()` and the report functions, are plain Python functions rather than routes.

Database access. `get_db()` opens one SQLite connection per request and stores it in Flask's `g` object; `close_db()` closes it when the request ends. Each connection runs `PRAGMA foreign_keys = ON`, because SQLite does not enforce foreign keys unless asked.

Security basics. Passwords are stored only as salted hashes (`generate_password_hash` [3]). Every SQL value is passed through a `?` placeholder, which prevents SQL injection. Jinja2 escapes all output, which prevents HTML injection. Every action that changes data is a `POST` form, and the session cookie uses `SameSite=Lax`, so browsers do not send it with form submissions coming from other websites. This blocks most cross-site request forgery; proper CSRF tokens are listed as future work.

4 Database design

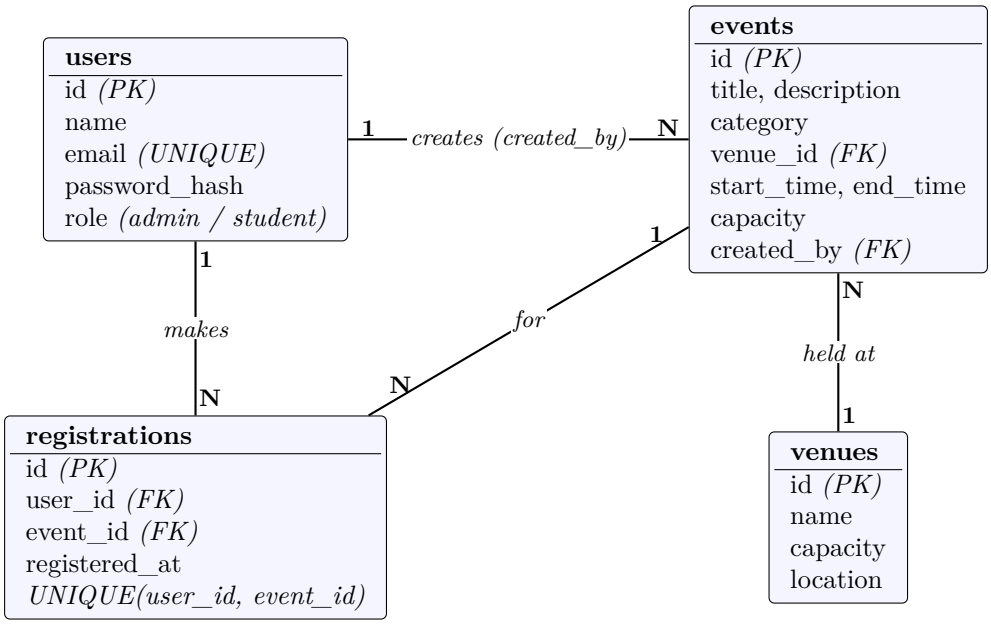


Figure 2: Entity–relationship diagram. `registrations` is the link table for the many-to-many relationship between students and events.

The schema (`sql/schema.sql`) has four tables:

- **users**: everyone who can log in. `email` is `UNIQUE`, and `role` has a `CHECK (role IN ('admin','student'))` constraint, so the database itself rejects any other value.
- **venues**: physical rooms, with `CHECK (capacity > 0)`.
- **events**: each event belongs to one venue (`venue_id`) and one creator (`created_by`). A table-level `CHECK (end_time > start_time)` makes events that end before they start impossible.
- **registrations**: the link table between students and events. `UNIQUE (user_id, event_id)` means a student can register for an event only once, even if the application code had a bug. `ON DELETE CASCADE` removes an event's registrations automatically when the event is deleted.

```
CREATE TABLE registrations (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL REFERENCES users(id)  ON DELETE CASCADE,
    event_id      INTEGER NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    registered_at TEXT    NOT NULL DEFAULT (datetime('now', 'localtime')),
    UNIQUE (user_id, event_id)   -- a student can register only once per event
);
```

Listing 1: The registrations table in sql/schema.sql.

Dates as text. Times are stored as `TEXT` in ISO format `'YYYY-MM-DD HH:MM'`. Because the most significant part comes first, comparing two such strings alphabetically gives the same answer as comparing the times. So `<`, `>` and `ORDER BY` work directly, which the clash check relies on.

Indexes. The schema deliberately started without extra indexes. They were added later in a separate file, `sql/indexes.sql`, as the result of profiling (Section 8).

5 Features and implementation

5.1 Authentication and roles

Students register through a form; the role is always fixed to `'student'` in the `INSERT`, so nobody can sign up as an admin (the admin account is created by `setup.sh`). A duplicate email raises `sqlite3.IntegrityError` from the `UNIQUE` constraint, which is caught and shown as a friendly message. Login uses `check_password_hash`, and the same message is shown for an unknown email and a wrong password so that attackers cannot find out which emails exist. Flask-Login keeps the user's id in a signed session cookie and turns it back into a `User` object on each request.

Admin-only pages are protected by a small decorator:

```
def admin_required(view):
    @functools.wraps(view)  # keep the original function's name for Flask's routing
    def wrapped_view(*args, **kwargs):
        if not current_user.is_authenticated:
            return login_manager.unauthorized()   # -> redirect to login page
        if not current_user.is_admin:
            abort(403)                            # logged in, but not an admin
        return view(*args, **kwargs)
    return wrapped_view
```

Listing 2: The admin_required decorator in app/auth.py.

5.2 Browsing and searching events

The events page lists upcoming events with the venue name and the number of seats left (`capacity - COUNT(r.id)`). The search form has a keyword, a category dropdown, a venue dropdown and a date range. It uses `GET`, so filters appear in the URL and a search can be bookmarked. The `WHERE` clause is built from two lists: `conditions` only ever holds fixed SQL text written by the programmer, and `params` holds the user's input, which reaches SQLite only through `?` placeholders.

```
conditions = ["e.start_time >= ?"]   # only upcoming events
params = [now_string()]
if keyword:
    conditions.append("(e.title LIKE ? OR e.description LIKE ?)")
    pattern = f"%{keyword}%"         # % means "any text" in LIKE
    params.extend([pattern, pattern])
if category:
    conditions.append("e.category = ?")
    params.append(category)
# ... venue_id, date_from and date_to are added the same way ...
sql = ("SELECT e.id, e.title, ..., e.capacity - COUNT(r.id) AS seats_left "
       "FROM events e JOIN venues v ON v.id = e.venue_id "
       "LEFT JOIN registrations r ON r.event_id = e.id "
       "WHERE " + " AND ".join(conditions) + " "
       "GROUP BY e.id ORDER BY e.start_time")
```

Listing 3: Building the search query safely (app/events.py, shortened).

Typing `' OR 1=1 -` into the search box therefore finds nothing: it is treated as ordinary text to search for, not as SQL.

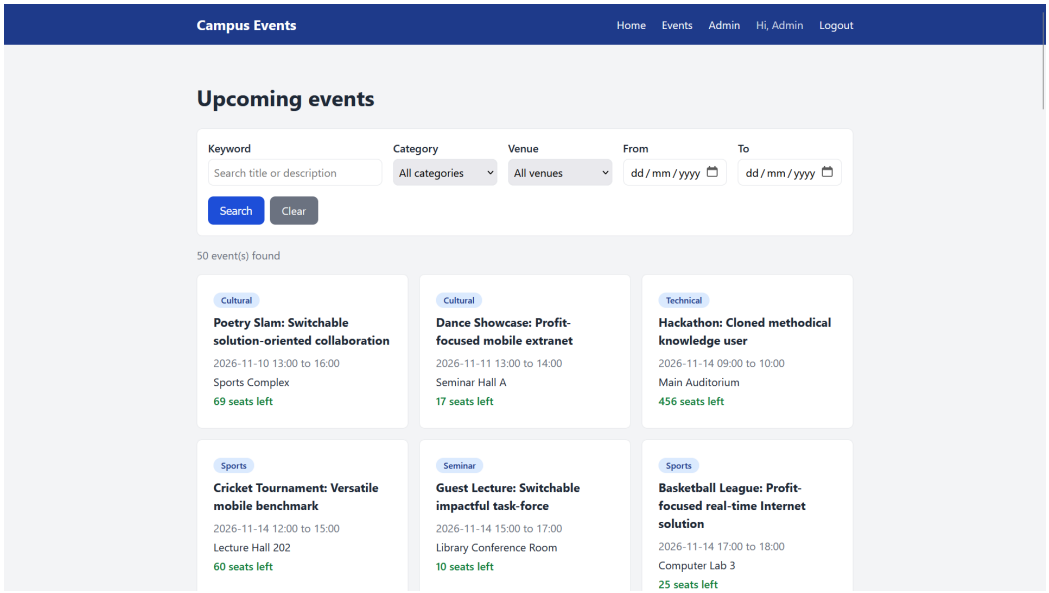


Figure 3: Searching and filtering upcoming events.

5.3 Registering and cancelling

Registration is a `POST` request from the button on the event detail page. The route checks every reason to refuse, in order, and explains which one applied:

```
if current_user.is_admin:
    flash("Admins cannot register for events.", "error")
elif event["start_time"] <= now_string():
    flash("This event has already started.", "error")
elif is_registered(current_user.id, event_id):
    flash("You are already registered for this event.", "error")
elif event["seats_left"] <= 0:
    flash("Sorry, this event is full.", "error")
else:
    conn.execute("INSERT INTO registrations (user_id, event_id) VALUES (?, ?)",
                 (current_user.id, event_id))
    conn.commit()
    flash(f"You are registered for {event['title']}.", "success")
```

Listing 4: Registration rules in register_for_event() (app/events.py).

Cancelling runs `DELETE FROM registrations WHERE user_id = ? AND event_id = ?` and uses `cursor.rowcount` to tell whether anything was actually deleted. The **My registrations** page joins `registrations`, `events` and `venues` for the logged-in student's upcoming events.

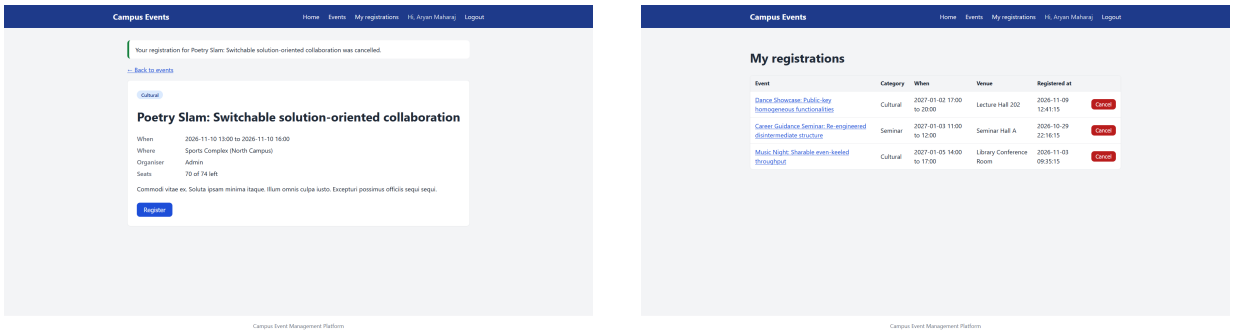


Figure 4: Event detail page (left) and a student's registrations (right).

5.4 Admin: venues, events and the clash check

Admins can create, edit and delete venues and events. New and edit share one form and one validation function, `read_event_form()`, which checks that the title and category are present, the times are valid, the event ends after it starts, the capacity is positive and not larger than the venue's capacity, and (when editing) not smaller than the number of students already registered. Finally it calls the clash check:

```
def find_clash(venue_id, start_time, end_time, ignore_event_id=None):
    """Return the first event at this venue whose time overlaps, or None.
    Two ranges overlap when each one starts before the other one ends."""
    sql = ("SELECT id, title, start_time, end_time FROM events "
           "WHERE venue_id = ? AND start_time < ? AND end_time > ?")
    params = [venue_id, end_time, start_time]
    if ignore_event_id is not None:      # when editing, skip the event itself
        sql += " AND id != ?"
        params.append(ignore_event_id)
    return get_db().execute(sql + " LIMIT 1", params).fetchone()
```

Listing 5: Venue clash check in app/admin.py.

Two events that only touch (one ends at 11:00, the next starts at 11:00) do not clash, because the comparisons are strict. When an event is edited, its own id is excluded; otherwise every edit would clash with itself. The form uses HTML `datetime-local` inputs, whose `2026-11-20T14:30` value is converted to the database format by `to_db_time()`.

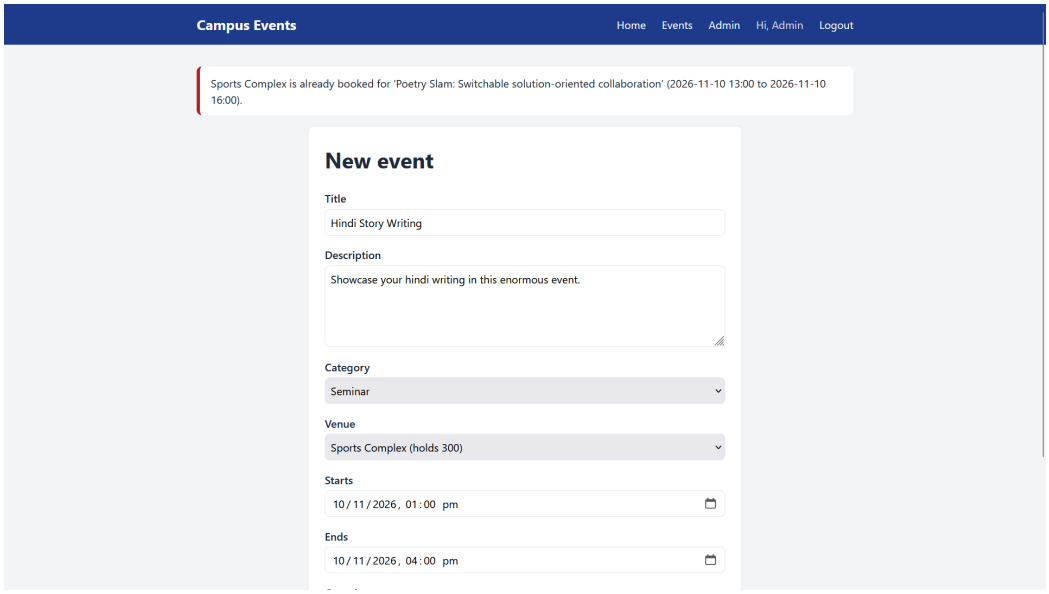


Figure 5: Creating an event that overlaps another event at the same venue is rejected.

5.5 Participants and CSV export

`/admin/events/<id>/participants` lists each participant's name, email and registration time. The **Export CSV** button returns the same rows written with Python's `csv.writer` into an in-memory `io.StringIO` buffer. A `Content-Disposition: attachment` header makes the browser download the file instead of displaying it. `csv.writer` takes care of commas and quotes inside names.

5.6 Reports

The reports page (Figures 6 and 7) shows popular categories, venue utilisation (registrations as a percentage of the seats offered by the venue's events) and registrations with fill rate for every event.

The calculations live in `app/reports.py` so that tests and the profiler can call them directly. How they were made fast is the subject of Section 8.

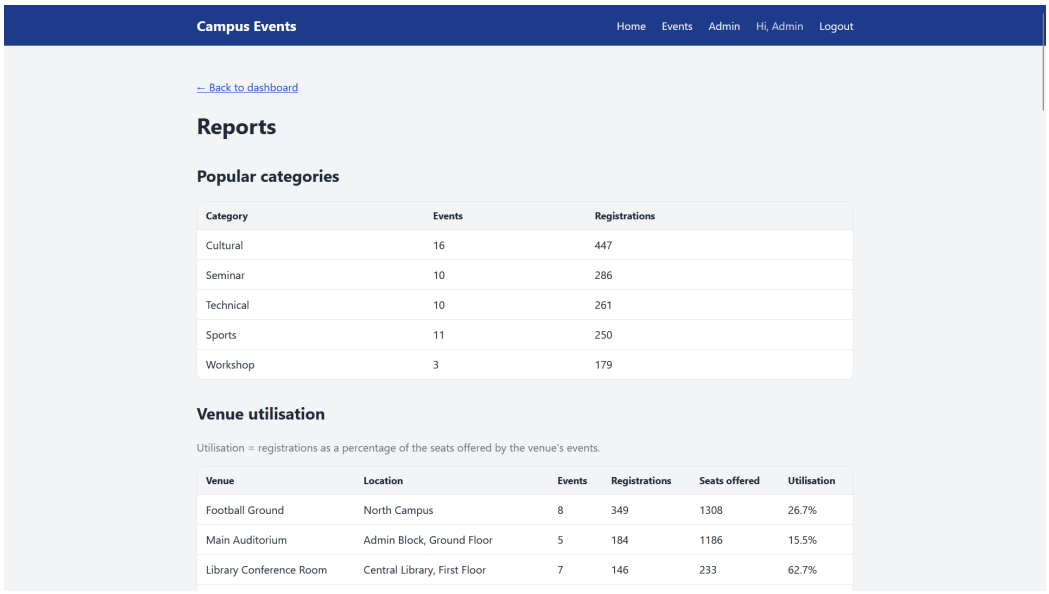


Figure 6: Reports page: popular categories and venue utilisation (demo data, 50 events).



Figure 7: Reports page: rest of venue utilisation and registrations per event with CSS fill-rate bars.

6 Bash automation

Every routine task has a script [13]. All of them start with `set -euo pipefail`: `-e` stops on the first failing command, `-u` treats unset variables as errors, and `pipefail` makes a pipeline fail if any part fails. They share helper functions from `scripts/common.sh`.

Script	What it does
`setup.sh`	Finds a working Python, creates `venv/`, installs requirements, creates the database, adds indexes, creates the admin (login can be changed with `ADMIN_EMAIL` / `ADMIN_PASSWORD`).
`run.sh`	Activates the venv, checks the database exists, starts Flask in debug mode (`PORT` can be changed).
`seed.sh [N]`	Validates N with a regular expression (`^[0-9]+$`) and fills the database with Faker [10] data.
`test.sh`	Runs `pytest -v`; extra arguments are passed on with `"$@"`.
`profile.sh`	Builds a separate profiling database (`CAMPUS_DB`), seeds it, optionally adds indexes (`APPLY_INDEXES`), runs the profiler and saves the query plans.
`build_docs.sh`	Compiles this report and the slides if LaTeX is installed.
`common.sh`	Shared functions `find_python` and `activate_venv`.

Table 2: Automation scripts in `scripts/`.

The scripts had to work in Git Bash on Windows as well as on Linux/macOS. Two details matter: a Windows virtual environment keeps its activate script in `venv/Scripts/` instead of `venv/bin/`, and on Windows `python3` can be a Microsoft Store shortcut that exists but only prints an error. So the helper actually *runs* each candidate:

```
find_python() {
    if python3 --version >/dev/null 2>&1; then
        PYTHON="python3"
    elif python --version >/dev/null 2>&1; then
        PYTHON="python"
    else
        echo "ERROR: Python 3 was not found."; exit 1
    fi
}

activate_venv() {
    if [ -f "$VENV_DIR/Scripts/activate" ]; then      # Windows
        ACTIVATE="$VENV_DIR/Scripts/activate"
    elif [ -f "$VENV_DIR/bin/activate" ]; then        # Linux / macOS
        ACTIVATE="$VENV_DIR/bin/activate"
    else
        echo "ERROR: run ./scripts/setup.sh first."; exit 1
    fi
    set +u; source "$ACTIVATE"; set -u    # activate reads unset variables
}
```

Listing 6: Finding a working Python and activating the venv (scripts/common.sh).

7 Testing

The project has **45 pytest tests, all passing** [9]. A fixture in `tests/conftest.py` creates a fresh app for every test with its database in pytest's temporary folder (`tmp_path`) and runs `schema.sql` on it, so tests never touch real data and cannot affect each other. Other fixtures provide a fake browser (`client`), helpers to register and log in, clients already logged in as a student or an admin, and a small set of sample events (three upcoming, one in the past).

Category (file)	Tests	What is checked
Authentication (`test_auth.py`)	7	register stores a hashed student, duplicate email rejected, correct and wrong password, student gets 403 on admin pages, admin allowed, logged-out user redirected to login
Browsing (`test_events.py`)	5	only upcoming events shown, keyword search (title and description), category filter, seats left, 404 for a missing event
Registrations (`test_registrations.py`)	8	register, double registration, full event, started event, cancel, My registrations, redirect after cancel, login required
Admin (`test_admin.py`)	18	`find_clash()` on its own (overlap, inside, touching, other venue, ignore itself), create event and time format, clash rejected, edit does not clash with itself, capacity above venue rejected, end before start, delete cascades, venues, students blocked from 7 admin URLs, CSV rows
Reports (`test_reports.py`)	7	`percent()`, exact numbers for all three reports, identical results with and without indexes, reports page, student blocked
Total	**45**	

Table 3: Test categories.

```python
def test_venue_clash_rejected(app, admin_client, sample_events):
    workshop = get_event(app, sample_events["Python Workshop"])
    before = count_events(app)
    # A new event at the same venue and the same time as the workshop.
    response = admin_client.post("/admin/events/new",
                                 data=event_form(workshop, title="Clashing Event"))
    assert b"already booked" in response.data
    assert count_events(app) == before
```

Listing 7: A test: a clashing event is rejected and nothing is saved (tests/test_admin.py).

The report tests are important for the optimisation: they check the exact numbers, not how they are computed, so the same tests passed unchanged before and after the reports were rewritten. That shows the faster version gives the same answers. `@pytest.mark.parametrize` runs one test for seven admin URLs, checking a student gets 403 on each.

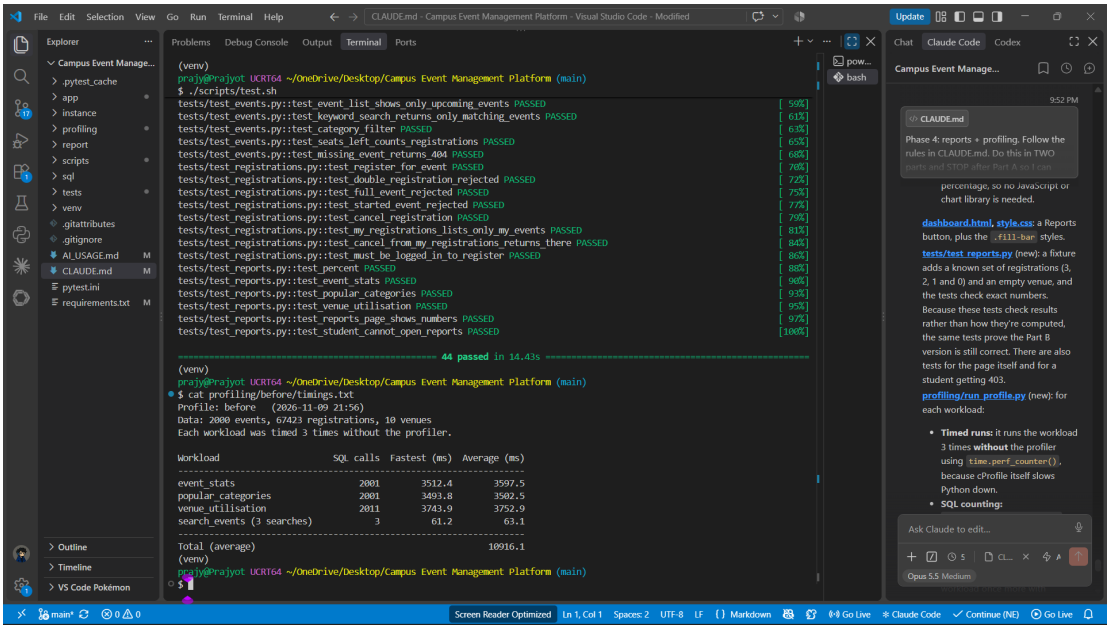


Figure 8: Running `./scripts/test.sh` in the VS Code terminal (Git Bash) (taken during the profiling phase, when there were 44 tests), followed by the baseline `timings.txt`.

8 Profiling and optimisation

8.1 Method

The reports page was first written the naive way on purpose, and then measured. `scripts/profile.sh` builds a separate database (`profiling/profile.db`, selected with the `CAMPUS_DB` environment variable so the demo data is never touched) and seeds it with **2000 events, 67,423 registrations, 10 venues and 100 students** using fixed random seeds, so every run uses the same data. `profiling/run_profile.py` then, for each workload:

1. runs it three times *without* the profiler and records the wall-clock time with `time.perf_counter()` (cProfile itself slows Python down);
2. counts the SQL statements one run sends, using `sqlite3`'s `set_trace_callback`;
3. runs it once more under `cProfile` [7] and saves `profile.prof` plus a `pstats` summary sorted by cumulative time.

The workloads are the three report functions and three typical event searches. `profiling/explain.py` also records `EXPLAIN QUERY PLAN` [4] for the important queries.

8.2 The bottleneck: the N+1 query problem

The naive report code fetched all events and then ran one `COUNT` query per event:

```
def count_registrations(event_id):
    """How many students registered for one event. ONE SQL query per call."""
    return get_db().execute(
        "SELECT COUNT(*) FROM registrations WHERE event_id = ?", (event_id,)
    ).fetchone()[0]

def event_stats():
    events = get_db().execute("SELECT id, title, ... FROM events").fetchall()
    for event in events:
        registered = count_registrations(event["id"])    # N+1: one query per event
```

```
    ...
```

Listing 8: Naive version (first commit of app/reports.py).

The top of the `cProfile` output made the problem obvious:

```
         86185 function calls in 10.646 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     6000    0.020    0.000   10.540    0.002 reports.py:21(count_registrations)
     6016   10.479    0.002   10.479    0.002 {method 'execute' of 'sqlite3.Connection'}
        1    0.010    0.010    3.574    3.574 reports.py:28(event_stats)
        1    0.005    0.005    3.511    3.511 reports.py:75(venue_utilisation)
        1    0.008    0.008    3.496    3.496 reports.py:56(popular_categories)
        3    0.000    0.000    0.063    0.021 events.py:52(search_events)
```

`count_registrations` was called 6000 times (2000 per report) and accounts for 10.54 of the 10.65 seconds, which is 99%. Almost all of that is *tottime* inside SQLite's `execute`, not in our Python code. The problem was not slow Python. It was **too many queries, each doing too much work**. `EXPLAIN QUERY PLAN` showed why each query was slow:

```
SCAN registrations USING COVERING INDEX sqlite_autoindex_registrations_1
```

The only index on `registrations` came from `UNIQUE(user_id, event_id)`. It starts with `user_id`, so it cannot help `WHERE event_id = ?`, and SQLite read all 67,423 entries for every single count: roughly $2000 \times 67{,}423 \approx 135$ million entries per report.

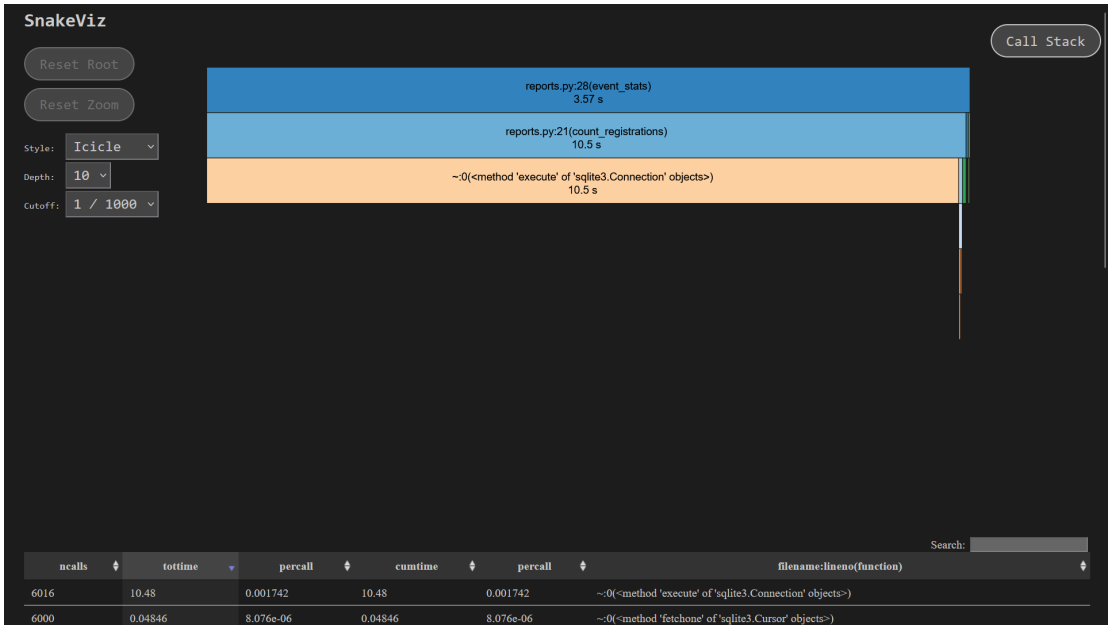
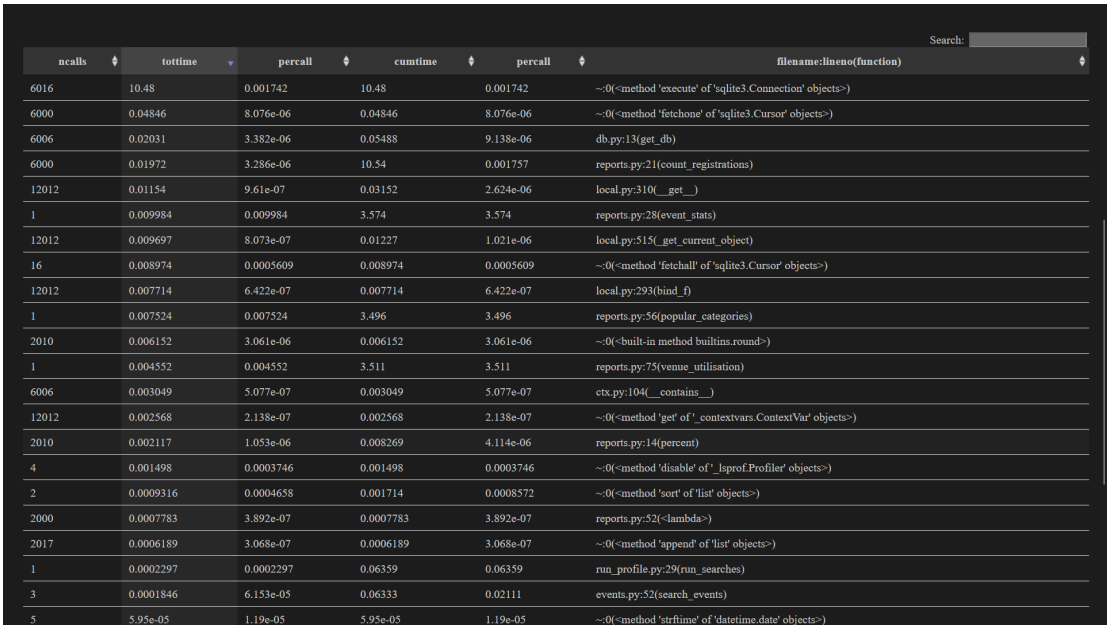


Figure 9: SnakeViz [8] icicle chart of the *before* profile: `event_stats` (3.57 s) is almost entirely `count_registrations` → `sqlite3 execute` (10.5 s in total).



Search:

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
6016	10.48	0.001742	10.48	0.001742	~:0(<method 'execute' of 'sqlite3.Connection' objects>)
6000	0.04846	8.076e-06	0.04846	8.076e-06	~:0(<method 'fetchone' of 'sqlite3.Cursor' objects>)
6006	0.02031	3.382e-06	0.05488	9.138e-06	db.py:13(get_db)
6000	0.01972	3.286e-06	10.54	0.001757	reports.py:21(count_registrations)
12012	0.01154	9.61e-07	0.03152	2.624e-06	local.py:310(__get__)
1	0.009984	0.009984	3.574	3.574	reports.py:28(event_stats)
12012	0.009697	8.073e-07	0.01227	1.021e-06	local.py:515(_get_current_object)
16	0.008974	0.0005609	0.008974	0.0005609	~:0(<method 'fetchall' of 'sqlite3.Cursor' objects>)
12012	0.007714	6.422e-07	0.007714	6.422e-07	local.py:293(bind_f)
1	0.007524	0.007524	3.496	3.496	reports.py:56(popular_categories)
2010	0.006152	3.061e-06	0.006152	3.061e-06	~:0(<built-in method builtins.round>)
1	0.004552	0.004552	3.511	3.511	reports.py:75(venue_utilisation)
6006	0.003049	5.077e-07	0.003049	5.077e-07	ctx.py:104(__contains__)
12012	0.002568	2.138e-07	0.002568	2.138e-07	~:0(<method 'get' of '_contextvars.ContextVar' objects>)
2010	0.002117	1.053e-06	0.008269	4.114e-06	reports.py:14(percent)
4	0.001498	0.0003746	0.001498	0.0003746	~:0(<method 'disable' of '_lsprof.Profiler' objects>)
2	0.0009316	0.0004658	0.001714	0.0008572	~:0(<method 'sort' of 'list' objects>)
2000	0.0007783	3.892e-07	0.0007783	3.892e-07	reports.py:52(<lambda>)
2017	0.0006189	3.068e-07	0.0006189	3.068e-07	~:0(<method 'append' of 'list' objects>)
1	0.0002297	0.0002297	0.06359	0.06359	run_profile.py:29(run_searches)
3	0.0001846	6.153e-05	0.06333	0.02111	events.py:52(search_events)
5	5.95e-05	1.19e-05	5.95e-05	1.19e-05	~:0(<method 'strftime' of 'datetime.date' objects>)

Figure 10: SnakeViz table of the *before* profile, sorted by own time: 6016 calls to `execute` take 10.48 s.

8.3 The fixes

1. One query per report. Each report became a single query using `LEFT JOIN` and `GROUP BY`, so the database counts everything in one pass. `LEFT JOIN` keeps events with no registrations (their count is 0). For the categories report, `COUNT(DISTINCT e.id)` is needed because the join repeats each event once per registration. For venue utilisation, an inner query first counts registrations per event; otherwise `SUM(capacity)` would be multiplied by the number of registrations. Percentages are still rounded in Python by the same `percent()` function, because Python and SQLite round halves differently.

```
SELECT e.id, e.title, e.category, e.start_time, e.capacity,
       COUNT(r.id) AS registered
FROM events e
LEFT JOIN registrations r ON r.event_id = e.id
GROUP BY e.id
ORDER BY registered DESC, e.start_time, e.id
```

Listing 9: Optimised event_stats query (app/reports.py).

2. Indexes. `sql/indexes.sql` adds three indexes [5], applied by `setup.sh` and `profile.sh`:

```
-- counting / joining registrations per event
CREATE INDEX IF NOT EXISTS idx_registrations_event ON registrations (event_id);
-- venue filter on the search page and the clash check
CREATE INDEX IF NOT EXISTS idx_events_venue_start ON events (venue_id, start_time);
-- category dropdown on the search page
CREATE INDEX IF NOT EXISTS idx_events_category    ON events (category);
```

Listing 10: sql/indexes.sql (comments shortened).

An index on `registrations(user_id)` was considered and *not* added: the `UNIQUE(user_id, event_id)` constraint already gives SQLite an index whose leftmost column is `user_id`, and the query plan for `WHERE user_id = ?  AND event_id = ?` already uses it. A second index would only slow down writes.

8.4 Results

	SQL calls		Average time (ms)			
Workload	**before**	**after**	**before**	**rewrite only**	**after**	**Speed-up**
`event_stats`	2001	1	3597.5	22.9	9.1	≈ 395×
`popular_categories`	2001	1	3502.5	36.0	6.5	≈ 539×
`venue_utilisation`	2011	1	3752.9	19.8	6.0	≈ 625×
`search_events` (3 searches)	3	3	63.1	66.0	10.9	≈ 5.8×
Total	**6016**	**6**	**10916.1**	**144.7**	**32.6**	**≈ 335×**

Table 4: Before and after, 2000 events and 67,423 registrations. *Rewrite only* = new queries without indexes (`APPLY_INDEXES=0 ./scripts/profile.sh rewrite_only`). Totals are computed from unrounded measurements.

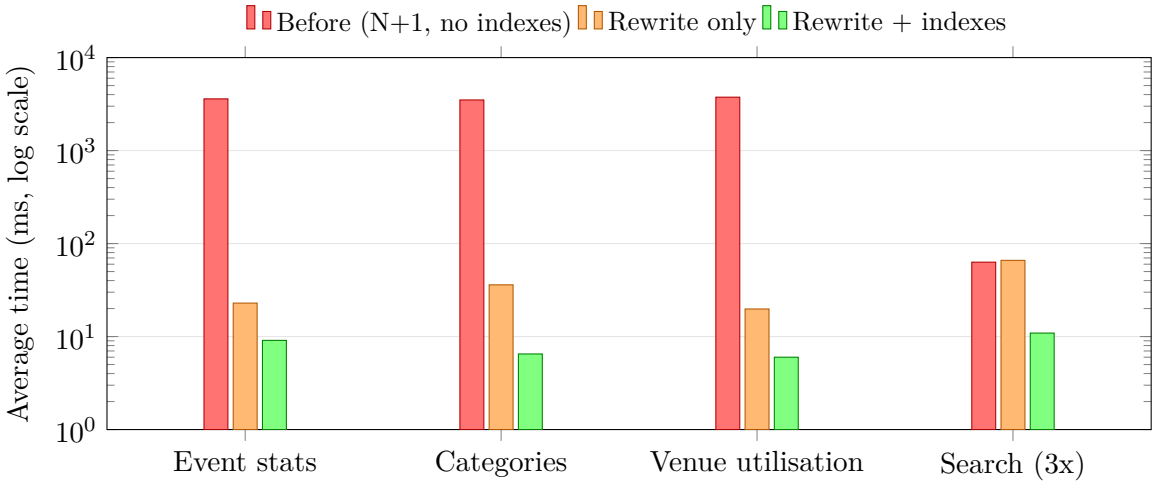


Figure 11: Average time per workload. Note the logarithmic scale: each grid line is 10×.

After the changes, `execute` is called 6 times and takes 0.026 s in total (it was 6016 calls and 10.48 s), and the profiled run makes 6,139 function calls instead of 86,185. An extra measurement of the *old* N+1 loop on the *indexed* database took about 115 ms. So for `event_stats` the indexes alone give about 31×, the rewrite alone about 157×, and both together about 395×. Each fix helps, and they work better together.

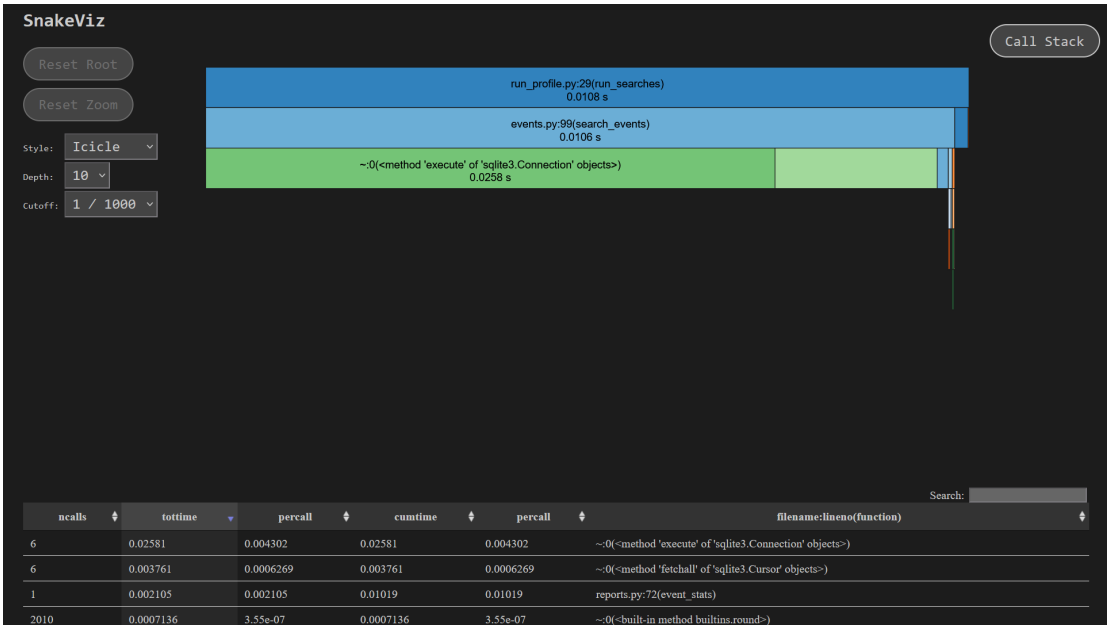


Figure 12: SnakeViz view of the *after* profile.

8.5 Query plans

`EXPLAIN QUERY PLAN` shows how SQLite runs a query. `SCAN` means reading every row; `SEARCH ... USING INDEX` means jumping straight to the matching rows; `AUTOMATIC ... INDEX` means SQLite had to build a temporary index for this one query and throw it away afterwards, which is a sign that a permanent index is missing.

Event search, no filters (the main events page):

```
BEFORE                                  AFTER
SCAN e                                  SCAN e
SEARCH v USING INTEGER PRIMARY KEY      SEARCH v USING INTEGER PRIMARY KEY
BLOOM FILTER ON r (event_id=?)
SEARCH r USING AUTOMATIC COVERING INDEX SEARCH r USING COVERING INDEX
         (event_id=?) LEFT-JOIN                  idx_registrations_event
                                                 (event_id=?) LEFT-JOIN
USE TEMP B-TREE FOR ORDER BY            USE TEMP B-TREE FOR ORDER BY
```

Before, SQLite built a temporary index over all 67,000 registrations *on every search*. After, it uses the permanent index, which explains 63 ms → 11 ms.

Other queries:

```
Per-event COUNT (naive report, run ~2000 times):
  before: SCAN registrations USING COVERING INDEX sqlite_autoindex_registrations_1
  after:  SEARCH registrations USING COVERING INDEX idx_registrations_event (event_id=?)
Search with category + date range:
  before: SCAN e
  after:  SEARCH e USING INDEX idx_events_category (category=?)
Search with venue filter:
  before: SCAN e
  after:  SEARCH e USING INDEX idx_events_venue_start (venue_id=? AND start_time>?)
Venue clash check:
  before: SCAN events
  after:  SEARCH events USING INDEX idx_events_venue_start (venue_id=? AND start_time<?)
```

8.6 Why it got faster, and what did not change

1. **Fewer round trips.** Every `execute()` has a fixed cost (Python to SQLite, preparing the statement, building result objects). 6016 calls became 6.
2. **Less data read per query.** An index is a sorted copy of a column with pointers back to the rows, like the index of a book. With it, SQLite finds one event's registrations in $O(\log n)$ steps instead of reading all n rows.
3. **No throw-away work.** The temporary `AUTOMATIC` indexes are no longer rebuilt for each query.
4. **Covering index.** `idx_registrations_event` contains everything a count needs, so SQLite never reads the registrations table itself.

Some things did not change. A keyword search still scans all events, because `LIKE '%workshop%'` starts with a wildcard and an ordinary index cannot help (like looking up every word that *contains* "shop" in a dictionary); SQLite's full-text search (FTS5) would be needed. Indexes are not free either: every `INSERT` or `DELETE` must also update them and the database file grows a little. Because this app reads far more often than it writes, the trade-off is worth it.

9 Git workflow

The project was developed in phases [11], with all commits made by the author on the `main` branch. Each commit is small, does one thing, and leaves the project in a working state where the tests pass. Commit messages follow the Conventional Commits style [12]:

Prefix	Meaning	Commits
`feat:`	a new feature	10
`test:`	adding or changing tests	3
`perf:`	a performance improvement	2
`chore:`	setup, tooling, scripts	3
`docs:`	documentation, notes and results	5
Total (before this documentation phase)		**23**

Table 5: Commit types.

Phase	Commits	Example commit messages
1. Skeleton	5	`chore: set up project structure and dependencies; feat: add SQLite schema for users, venues, events, registrations`
2. Auth and browsing	6	`feat: add student registration, login and logout with Flask-Login; test: add pytest fixtures and auth/event tests`
3. Registrations and admin	5	`feat: let students register for and cancel events; feat: add admin CRUD for venues and events with clash check and CSV export`
4a. Naive reports + baseline	3	`feat: add admin reports page (naive N+1 version) with tests; docs: record baseline profile of naive reports`
4b. Optimisation	4	`perf: replace N+1 report loops with JOIN + GROUP BY queries; perf: add database indexes and apply them in setup and profiling`
Total	**23**	

Table 6: History summary (`git log -oneline`).

Two habits made the history useful. First, the naive reports and their baseline measurements were committed *before* the optimisation, so the history itself proves what was measured and when. Second, `.gitignore` keeps build and environment files (the virtual environment, `*.db`, `__pycache__`, LaTeX build files) out of the repository, and `.gitattributes` forces `.sh` files to use Unix line endings, so a Windows checkout cannot break the Bash scripts with `$'\r': command not found`.

10 AI usage declaration

I used Claude, an AI assistant made by Anthropic (through the Claude Code extension in VS Code), during this project.

What it was used for. Drafting much of the application code, Bash scripts, tests and documentation from my specifications (the features, the database schema, raw SQL without an ORM, and a naive first version of the reports to be profiled and then optimised); explaining concepts and code, such as Flask Blueprints, cProfile output, query plans and indexes; and helping diagnose problems and suggest improvements, including the Windows `flask.exe` launcher issue, the Jinja comment bug and the redundant `registrations(user_id)` index.

What I did. I defined the requirements and the order of work. I ran every script and the full test suite myself in Git Bash on Windows and tested the features in the browser. I reviewed all of the code and every suggestion before accepting it, including the decision to leave out the redundant index. I ran the profiling, took the screenshots and made every git commit myself. A phase-by-phase log is in `AI_USAGE.md` in the repository.

In my own words. Working on this project helped me understand how different components of a web application work together, particularly Flask routing, database operations using SQLite, authentication, and event registration workflows. I also gained practical experience in writing Bash automation scripts, testing application functionality with pytest, and analysing performance using cProfile and SQLite's `EXPLAIN QUERY PLAN`. I used AI tools, including Claude and Claude Code, to assist with code drafting, debugging, documentation, and understanding unfamiliar concepts. However, I reviewed and modified the generated code, executed the application and

tests locally, verified the reported profiling results, and checked the final outputs myself. This project helped me understand the importance of independently validating AI-assisted work rather than relying on generated code without verification.

11 Challenges

Windows: `flask.exe` "Permission denied". The first run of `setup.sh` failed when Git Bash tried to start the `flask.exe` launcher inside the virtual environment. Running Flask as a module, `python -m flask -app app ...`, avoids the launcher entirely and works the same on every operating system, so every script uses it. A related trap was that `python3` on Windows can be a Microsoft Store shortcut that exists but does not work, which is why `find_python` runs `-version` instead of only checking that the command exists.

Jinja parses tags inside HTML comments. The base template contained the HTML comment `<!-- Each page can set its own title with {% block title %} -->`. The browser would ignore it, but Jinja processes the template first, saw a real `{% block %}` tag and failed with "Unexpected end of template". The fix is to use Jinja's own comment syntax, `{# ... #}`, for comments that mention template tags.

Which indexes to add. The plan listed an index on `registrations(user_id)`. Checking the query plans showed that the `UNIQUE(user_id, event_id)` constraint already creates an index starting with `user_id`, which the "is this student registered?" query already uses. Adding another would cost disk space and slow down every registration while making no query faster, so it was left out and the reason documented in `sql/indexes.sql`.

The last-seat race condition. Registration checks the seats left and then inserts in two separate statements. If two students click *Register* for the last seat at almost the same moment, both checks could see one seat left before either insert happens, and the event would be over-booked by one. At demo scale this is practically impossible, but the proper fix is to make the check and the insert one atomic step: for example, a single `INSERT ... SELECT ... WHERE (SELECT COUNT(*) ...) < capacity` and checking whether a row was inserted, or wrapping both in a `BEGIN IMMEDIATE` transaction.

A failing test that was the test's fault. The first version of the clash-check test used a time window starting in the year 2000 and expected no clash. It failed, because that window really did overlap a past event at the same venue in the sample data. The application was right and the test was wrong; the test now uses a one-hour window just before the event.

12 Conclusion and future work

The Campus Event Management Platform meets its objectives. Students can find and register for events, admins can manage events and venues without double-booking a room, and reports show how events and venues are used. The project combines HTML/CSS, Python, SQL, Git, Bash and LaTeX; it is covered by 45 automated tests; and profiling found a real bottleneck, the N+1 query problem made worse by a missing index, whose fix made the reports about 335 times faster without changing a single reported number.

The main lesson was to *measure before optimising*: the profiler showed that the cost was in the number and kind of SQL queries, not in Python, and `EXPLAIN QUERY PLAN` showed exactly which index was missing.

Future work.

- Make registration atomic to remove the last-seat race condition.
- CSRF tokens on forms (for example with Flask-WTF) in addition to `SameSite` cookies.
- Full-text search (SQLite FTS5) so keyword search can use an index.
- Pagination for the event list and reports, email confirmations and reminders, and a waiting list when an event is full.
- Deployment with a production WSGI server and a real secret key.

References

[1] Pallets Projects. *Flask documentation*. https://flask.palletsprojects.com/

[2] *Flask-Login documentation*. https://flask-login.readthedocs.io/

[3] Pallets Projects. *Werkzeug: security helpers*. https://werkzeug.palletsprojects.com/

[4] SQLite. *The SQLite query optimizer overview* and *EXPLAIN QUERY PLAN*. https://www.sqlite.org/optoverview.html, https://www.sqlite.org/eqp.html

[5] SQLite. *CREATE INDEX* and *automatic indexes*. https://www.sqlite.org/lang_createindex.html

[6] Python Software Foundation. *sqlite3: DB-API 2.0 interface for SQLite databases*. https://docs.python.org/3/library/sqlite3.html

[7] Python Software Foundation. *The Python profilers (cProfile, pstats)*. https://docs.python.org/3/library/profile.html

[8] *SnakeViz*. https://jiffyclub.github.io/snakeviz/

[9] *pytest documentation*. https://docs.pytest.org/

[10] *Faker documentation*. https://faker.readthedocs.io/

[11] S. Chacon and B. Straub. *Pro Git*, 2nd ed., Apress. https://git-scm.com/book

[12] *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/

[13] GNU. *Bash Reference Manual*. https://www.gnu.org/software/bash/manual/

[14] T. Tantau. *The TikZ and PGF Packages: Manual*. https://ctan.org/pkg/pgf